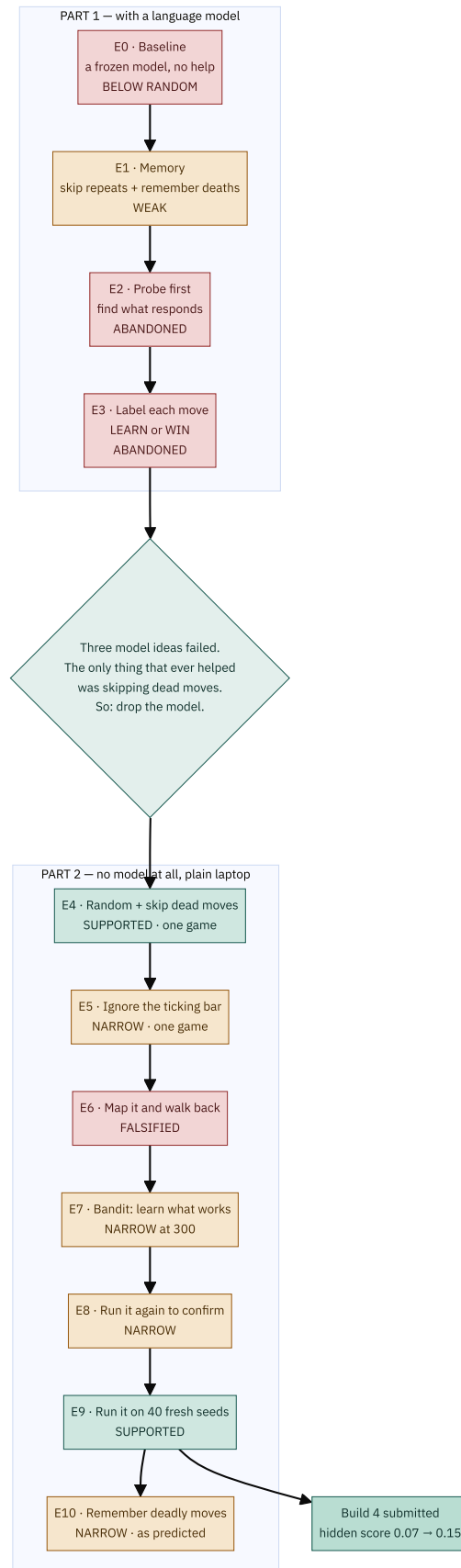


# Eleven experiments, and what each one taught us

E0 through E10. Every one had its falsifier written down before it ran. Most of them failed. The failures are what made the two that worked believable.

---

# The whole story in one picture



# Four words you need first

| WORD             | WHAT IT MEANS HERE                                                                                                      |
|------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Held-out</b>  | 17 games kept aside and only played at the very end. The verdict comes from these, never from the 8 practice games.     |
| <b>Seed</b>      | One repeat of the whole run with different random luck. 20 or 40 seeds, so one lucky run cannot decide anything.        |
| <b>Supported</b> | It beat its own pre-written test, <i>and</i> the gain did not come from one game.                                       |
| <b>Narrow</b>    | It beat the test, but remove the single best game and the win disappears. Counted as "not shown to work", not as a win. |

## Why "narrow" is treated as a failure

Early on, a doubled score turned out to come almost entirely from one level of one game. The total said "strong method"; the per-game table said "one game". Since then, every result has to survive deleting its best game. Most did not.

### PART 1

## Four tries with a language model

The plan was: give a small frozen model the screen, let it choose moves, and see whether adding memory or instructions makes it play better. The model's weights never changed. Only the harness around it changed.

## Can a frozen model play at all, with no help?

### WHAT WE DID

Show the screen, let the model pick a move, repeat. No memory, no rules. Run it at two sizes — a small model and one eight times bigger — on the 17 held-out games, 5 seeds each.

### WHAT WENT WRONG

**It cleared zero levels. At both sizes.** Random play cleared more. Making the model 8× bigger changed nothing on the main measure. It also cost real money and time: 90 seconds per decision at the large size, about 10 GPU-hours for one verdict.

### WHAT WENT RIGHT

We learned the official score is almost useless at this level — it reads 0.0000 on 23 of 25 games, so it cannot tell a good agent from a bad one down here. We switched to counting **levels reached** and **how many different screens were visited**, which do separate the games. The bigger model did halve one specific weakness: it wasted fewer clicks. Not enough to clear anything.

### SO WE MOVED ON

If a bare model is below random, the interesting question is whether a harness around it can lift it. That is E1.

## Does memory help — skipping repeats, and remembering how you died?

### WHAT WE DID

Two switches, tested together and separately. **Switch A:** notice you have been on this screen before and stop repeating moves known to do nothing. **Switch B:** when you die, write a rule so you do not die that way again.

### WHAT WENT RIGHT

**Switch A did something real.** The agent explored much more — it visited far more different screens, closing over half the gap to random play, and on two games it beat random outright.

### WHAT WENT WRONG

**Switch B did nothing at all.** 34 rules were written and none of them mattered — because the failure they target barely happens. In 85 attempts the agent died the same death twice exactly *once*. There was nothing to fix.

And the one level that got cleared was mostly luck: on 64% of turns the model's chosen click was dead and the harness silently swapped in a random live one. So the clear cannot be credited to the model.

### SO WE MOVED ON

Two lessons. The two switches are *not* one idea — they were merged on a reviewer's advice and that was wrong. And skipping dead moves helps **exploring**, not **solving**. Park it; test something else on the model.

## Does it help to spend the first few moves finding out what responds?

### WHAT WE DID

Before playing properly, spend the first 5, 10 or 20 moves deliberately poking at things to learn what reacts. Then tell the model what it found. Compared against a fair control that got the *same extra moves* with no instructions — so we were testing the probe, not just "more moves".

### WHAT WENT WRONG

**The result came out backwards.** The probed model wasted *more* of its moves than the plain one, at every setting. Not a small difference — at the shortest probe it was three times the control's own variation.

On one game it collapsed completely: after reading the probe summary, the model settled into pressing things that did nothing, on 90–96% of turns.

### WHAT WENT RIGHT

It stopped at the practice games and never touched the held-out 17. The stopping rule was written before the run and it fired. Cost: about 2.7 GPU-hours instead of a wasted week.

### SO WE MOVED ON

Honestly recorded as unresolved: we cannot tell whether the *idea* is wrong or our *probe* was badly built. Two specific suspects were named in advance and never tested. Next idea.

## Does it help if the model says why it is making each move?

### WHAT WE DID

Ask the model to tag every move as either **LEARN** (I am finding out what this does) or **WIN** (I am trying to progress) — the way a human learning a new game switches between the two.

### WHAT WENT WRONG

**Worse again**, and the same direction as E2: more wasted moves, fewer screens explored, zero levels cleared.

Worse for the test itself: **the model ignored the instruction on 67% of its replies**. Only about a fifth were tagged **LEARN** and an eighth **WIN**. So this is a weak test — the honest reading is "this model does not follow the instruction reliably", not "the idea is wrong".

And the same game that broke under E2 broke again here. Twice now, adding text to the prompt has pushed this model into repeating moves that do nothing.

### BONUS CATCH

The first attempt produced zero data — a missing file meant every game process died on startup. Six GPU-minutes lost. A check was added so a run now fails immediately if it ships code that imports a file it did not ship.

### SO WE MOVED ON

Three model experiments, three failures, all pointing the same way: *this frozen model does not turn more prompt structure into better play*.

### THE TURNING POINT

## Stop using the model. Test the one thing that actually worked.

Across E0–E3 exactly one component ever moved a number: E1's "skip moves known to do nothing". It helped the *exploring*, not the model's judgement.

So the question became: **does that trick help plain random play, with no model at all?**

That costs a few CPU-hours on a laptop instead of ten GPU-hours, and it is the only candidate that could improve the competition entry before the deadline.

Everything from here runs on an ordinary laptop. No GPU, no API key, no model.

## Random play that never repeats a move known to do nothing

### WHAT WE DID

Plain random play, with one rule: if a move already left this exact screen unchanged, draw a different one. 17 games × 20 seeds, at 80 and 300 moves per attempt.

### WHAT WENT RIGHT

**Twice the levels cleared — 38 against 19 — winning 18 seeds out of 20 and losing none.** The clearest result in the project so far.

### WHAT WENT WRONG

**One game carries all of it.** On [1p85](#) it clears level 1 in 20 seeds out of 20; plain random manages 1 in 20. On every other game the two are nearly identical.

It also barely moved the official score, because the clears are *slow* — 38 to 288 moves where a human needs 17 — and the score punishes slowness. More levels reached, not faster play.

Submitted as Build 3: hidden-set score **0.07 — exactly the same as plain random.** The public gain did not transfer.

### SO WE MOVED ON

Why does it only work on one game? Because the rule needs the screen to be *identical*, and most games have a bar or counter that changes on every single move. That is E5.

### The correction that matters most in this whole record

On 17 September, while checking why E4 showed levels being cleared in one move — which is impossible — a bug was found. The code built **one game world per game and replayed every attempt on it.** Clearing a level once meant every later attempt started past it and counted its first move as a clear.

This had inflated the random baseline that E0 and E1 had been measured against. Everything affected was re-run, the old numbers were **left visible with corrections written underneath**, and two sentences that had been stated confidently were formally withdrawn. A regression test now proves every attempt starts at level 1.

This is why the day-one gate checks "every episode starts at level 1". That check exists because of this bug.

## Ignore the ticking bar when deciding whether the screen changed

### WHAT WE DID

If a row or column of the screen changes on nearly every move, it is a progress bar, not a result. Cover it up before comparing screens. Then "did my move do anything?" can be answered honestly.

### WHAT WENT RIGHT

It did exactly what it was designed to do on the game it was designed for. On `vc33`, a click-only game with a bar on the top row, the old rule learned **zero** dead moves. The new rule learned 1,903, and went from **0 clears to 20 out of 20**.

Total clears rose 37 → 57, winning 18 seeds and losing none.

### WHAT WENT WRONG

**Remove `vc33` and it is 37 against 37.** Exactly one game improved. Narrow.

One game showed signs of over-masking — 16 columns covered up, which is almost certainly too much.

### SO WE MOVED ON

Not adopted as the way the agent *plays*. But adopted as the common **measuring tool** for every later experiment, since all of them need an honest answer to "did anything change?".

## Draw a map, and walk back to places with unexplored doors

### WHAT WE DID

Remember every screen and every move tried from it. When stuck, plan a route back to a screen with untried moves and go there. Compared against the same agent with the map switched off.

### WHAT WENT WRONG

**The map was almost never needed.** Navigation used only 4–6% of all moves, over routes averaging 1.5 steps. Within the move budget, the simple local rule hardly ever runs out of new things to try — so there is nothing to navigate back *for*.

The result failed its statistical test once corrected for testing two budgets. And the small gain that existed came from one game again.

### WHAT WENT RIGHT

A clean, useful negative: **re-reaching known places is not what limits these games at these budgets.** That removes a whole family of ideas from consideration.

A side finding worth more than the main one: the *way moves were grouped* for this experiment — one choice per colour on screen, instead of per pixel — beat the previous best by a wide margin on its own.

### SO WE MOVED ON

Keep the colour grouping, drop the map. If grouping moves by colour helps, then *learning which group pays off* is the obvious next thing. That is E7.

## Learn during the game which kinds of move pay off

### WHAT WE DID

Keep a running score for each kind of move — each button, and each colour you can click. Spend more of the remaining budget on the ones that change the screen. This is the "bandit" idea.

### WHAT WENT RIGHT

It clearly does what it claims: **wasted moves nearly halved** (30% → 14%), and the number of different screens visited rose from about 53 to about 68 per 100 moves. On one game it worked out that clicking one specific colour was the thing that mattered and put 85% of its effort there.

At the longer budget of 1,000 moves it was much stronger — and **it was the first result in the whole project to survive deleting its best game.**

### WHAT WENT WRONG

At the budget we had declared as primary (300 moves) it was **narrow** — one game carried it.

Two games got consistently *worse*. The suspected reason: the reward is "did the screen change?", so on those games the agent locks onto a move that changes the screen without making any progress.

### SO WE MOVED ON

The strong 1,000-move result was seen *after* declaring 300 primary, so it does not count as a verdict. Our own rule says: write a new plan and test it again on attempts never used before. That is E8.

## Run the same bandit again, properly, to confirm it

### WHAT WE DID

Same agent, no changes. This time 1,000 moves declared primary *in advance*, on 20 fresh attempts never played before.

### WHAT WENT RIGHT

**The size of the effect repeated almost exactly:** 117 vs 82 in E7, 114 vs 79 in E8. The wasted-move rate dropped identically both times (30% → 14%). The *same games* gained and the *same two games* lost.

### WHAT WENT WRONG

**The breadth did not repeat.** E7 survived deleting its best game; E8 did not. So by the rule written in advance, the confirmation run **did not confirm**.

### SO WE MOVED ON

A fair criticism of our own test: with 20 seeds, deleting the best game and re-testing is a very strict bar that may simply lack the power to detect a real effect. The fix is more seeds — not a change to the agent. That is E9.

## The same bandit on 40 fresh attempts

### WHAT WE DID

Identical agent, identical code. 40 attempts per game, all of them new. Nothing else changed — deliberately, so that only the number of seeds differs.

### WHAT WENT RIGHT

**217 clears against 171. 22 seeds won, 4 lost. And this time it survived deleting its best game.**

Seven games improved, not one. This became the project's new baseline: anything proposed from here on has to beat *this*, not random play.

**Submitted as Build 4 → hidden-set score 0.15, up from 0.07.** The first time in the project that a gain measured on the public games transferred to the 110 hidden ones.

### WHAT WENT WRONG

Three caveats were written down *with* the verdict, not afterwards:

- **It is the third test of the same idea**, and each was decided after seeing the last. A positive after a narrow result is weaker evidence than a first-time positive.
- **The breadth test barely passed** —  $p = 0.044$  against a 0.05 bar.
- **The same two games lose every single time**, in all three runs. The gain is net, not universal.

And the 0.15 cannot be split between the smarter policy and the raised move budget, because Build 4 changed both at once.

### SO WE MOVED ON

Two follow-ups. A second submission with the learning switched off, to separate policy from budget. And: *why do those two games always lose?* That question produced E10.

## Remember the move that killed you, on the screen it killed you on

### WHERE IT CAME FROM

We read the actual play logs of the two losing games. On one, the agent **dies in a loop**: a move that ends the game is never marked as dead, the reset returns to the same screen, and the learner — now confident — makes the same move again. 38% of its deaths repeated a screen it had already died on, against 3% for the simpler agent.

### WHAT WE DID

One rule, nothing else: if a move ends the game, do not use that move on *that screen* again this attempt. It stays available everywhere else.

### SAID IN ADVANCE

Two things were written down before running, and both came true:

- "The most likely outcome is a gain concentrated on **sp80**, i.e. narrow."
- "This design is adaptive — the fix was found by looking at these games, so a positive result here is weaker evidence than a first-time positive."

### WHAT WENT RIGHT

**The mechanism did exactly what the diagnosis said it would.** Repeat deaths across all games fell from 534 to 34. On the target game they fell from 403 to 2, and clears rose from 18 to 30.

It cost nothing elsewhere — every other game combined was 215 against 209.

### WHAT WENT WRONG

**Narrow.** Delete the target game and the win vanishes. So it is not adopted as the baseline.

The second losing game did not move at all — 8,570 pulls, 360 deaths, 11 clears, identical. It has no death loops. Its problem is a different mechanism entirely, still unsolved.

### WHERE IT STANDS

A fix that provably works on the thing it targeted, and is not shown to generalise. The only honest test left for it is the hidden set.

## What the hidden set has actually said

The 110 hidden games are the only truly unseen test. Scores come back to two decimal places, so anything smaller than 0.01 is invisible.

| BUILD   | WHAT IT WAS                              | SCORE   | READING                                                                                                                                                                                                         |
|---------|------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Build 1 | Plain random, 80 moves                   | 0.07    | The floor.                                                                                                                                                                                                      |
| Build 3 | E4's random + skip dead moves, 300 moves | 0.07    | No visible change. The public gain did not transfer.                                                                                                                                                            |
| Build 4 | E9's bandit, 1,000 moves                 | 0.15    | Doubled. The first real movement — but policy and budget changed together.                                                                                                                                      |
| Build 5 | Build 4 with the learning switched off   | pending | Built and pushed; the submission was blocked by the daily quota on 30 Sept. Its score decides whether the gain was the learning or the bigger budget — and how it will be read was written down before it runs. |

### WHAT IT ADDS UP TO

## Five things this record actually establishes

- **A frozen small model with a simple scaffold does not beat random play here** — not at 8B, not at 31B, not with memory, not with a probe, not with labels. Four separate tries.
- **The best policy we have contains no model at all.** It runs on a laptop in minutes. That is the uncomfortable finding, and it is the one worth publishing.
- **Skipping moves that do nothing is genuinely the core trick** — it turned up in E1, carried E4, needed E5's fix to work widely, and is inside every later agent.
- **Learning which kinds of move pay off is the one thing that transferred** to the hidden games. Everything else stalled at 0.07.
- **Most ideas fail, and saying so cheaply is the point.** E2, E3 and E6 were killed for a few hours of compute each because their stopping rules were written first.

## What nobody has answered yet

- **Why does `cd82` always lose?** Three runs, same loss, and E10's fix did not touch it. Something about that game defeats the reward.
- **Is "did the screen change?" the wrong reward?** It cannot tell progress from movement, which is the suspected cause of every loss we have.
- **How good could an agent be if it already knew the goal?** Nobody has measured the ceiling. That is P5, and it needs no code.
- **Does clicking visible objects beat clicking anywhere?** Being tested now as E11 — and independently as P2.

---

Every experiment above was written into the register with its falsifier before it ran. Where a result was later found to be wrong, the original is left visible and the correction is written underneath it. A research record in which nothing was ever wrong is a record nobody was checking.

# The scoreboard, completed

The record above was written on 1 October, when the last two submissions had not been scored. Both have now come back. Nothing earlier in the document is withdrawn — this page only completes the *“What the hidden set has actually said”* table, where Build 5 was listed as *pending*.

| Build   | What it was                                                                                               | Score | Reading                                                                                                                                                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Build 1 | Plain random, 80 moves                                                                                    | 0.07  | The floor.                                                                                                                                                                                                                        |
| Build 3 | E4's random + skip dead moves, 300 moves                                                                  | 0.07  | No visible change. The public gain did not transfer.                                                                                                                                                                              |
| Build 4 | E9's bandit, 1,000 moves                                                                                  | 0.15  | Doubled — but policy and budget changed together.                                                                                                                                                                                 |
| Build 5 | Build 4 with the learning switched off (uniform choice over the same move groups, same 1,000-move budget) | 0.12  | <b>Resolved.</b> Splits Build 4's gain: about +0.05 came from the move grouping, the masked judge and the larger budget together, and about <b>+0.03 from the learning itself</b> .                                               |
| Build 6 | Build 4 plus a per-screen memory of deadly moves (E10)                                                    | 0.15  | <b>No change.</b> How this would be read was fixed before the submission: 0.14–0.15 meant “adds nothing visible”. A fix that provably ended the death loops on the one game it targeted did not transfer to the 110 hidden games. |

## What the two new numbers add up to

- **The ladder stops at 0.15.** 0.07 → 0.07 → 0.15 → 0.15. Of the +0.08 over Build 3, roughly +0.05 is the move grouping, judge and budget, +0.03 is the learning, and +0.00 is the death memory.
- **Learning which kinds of move pay off remains the only thing that transferred.** Build 5 is what makes that claim measurable rather than asserted: without a control at a matched budget, the whole +0.08 would have been credited to the bandit.
- **A narrow result stayed narrow on unseen games.** E10 was recorded as carried by a single game, and the hidden set agreed. That is what a narrow verdict predicts, and it is why the rule was not promoted to baseline.
- **Both readings were written before the scores arrived.** Build 5's purpose and Build 6's three score bands were fixed in advance, so each number was a verdict on the day it landed rather than an argument afterwards.

Scores are public leaderboard results, each one submission reported to two decimal places, so a difference of 0.01 or less is not a difference. Builds 4, 5 and 6 differ from one another by exactly one change each, which is what makes them comparable.